

Documento Complementario: Herramientas de Ingeniería Aplicadas al Radar Acústico Monostático

José Andrés Solano Mora
Escuela de Ingeniería en Computadores
Tecnológico de Costa Rica
Cartago, Costa Rica

Resumen—Este documento técnico detalla la selección, aplicación y adaptación de las herramientas hardware, software, analíticas y de Inteligencia Artificial involucradas en la solución del sistema de radar acústico monostático. Se justifica la integración del microcontrolador ATmega328P, el entorno Python para procesamiento digital de señales (PDS) y el soporte de IA generativa (Gemini) en el diseño algorítmico y matemático, cumpliendo con la extensión máxima de 2 páginas.

I. SELECCIÓN DE TÉCNICAS, RECURSOS, HERRAMIENTAS O MÉTODOS

Para resolver la estimación de distancia por tiempo de vuelo (*Time of Flight*, ToF) considerando las restricciones físicas del sistema acústico, se seleccionaron las siguientes herramientas organizadas por dominio:

I-A. Plataforma de Hardware Empotrado

- **Microcontrolador ATmega328P (Arduino):** Seleccionado por su temporización por hardware precisa y control de periféricos mediante acceso a registros.
- **Transductor Piezoeléctrico Pasivo:** Utilizado como elemento emisor mediante excitación con modulación de frecuencia constante a 2 kHz.
- **Micrófono Electret con Preamplificador MAX4466:** Seleccionado como receptor debido a su ganancia ajustable y respuesta en frecuencia apta para la banda audible.

I-B. Entorno de Procesamiento Digital y Software

- **Lenguaje C/C++ (Arduino AVR):** Empleado para la adquisición determinista a la frecuencia de muestreo $F_s = 10$ kHz.
- **Python 3 (NumPy, PySerial, Matplotlib):** Utilizado para la captura serial, eliminación de sesgo DC, implementación del filtro adaptado y graficación en tiempo real.
- **Transformada Rápida de Fourier (FFT):** Método matemático seleccionado para realizar la correlación cruzada en el dominio de la frecuencia, reduciendo la complejidad computacional de $\mathcal{O}(N^2)$ a $\mathcal{O}(N \log_2 N)$.

I-C. Herramientas de Inteligencia Artificial y Guía

- **Modelo de Lenguaje (Gemini AI):** Utilizado como herramienta de asistencia técnica en la optimización de código en Python, derivación de ecuaciones de correlación cruzada y maquetación de informes técnicos en LaTeX.

II. APLICACIÓN DE LAS HERRAMIENTAS EN EL PROBLEMA COMPLEJO DE INGENIERÍA

El problema complejo radica en detectar un eco acústico atenuado e inmerso en ruido de acoplamiento directo utilizando un microcontrolador de 8 bits y transmisión de bajo ancho de banda. Las herramientas se aplican secuencialmente según el flujo funcional presentado en la Fig. ??.

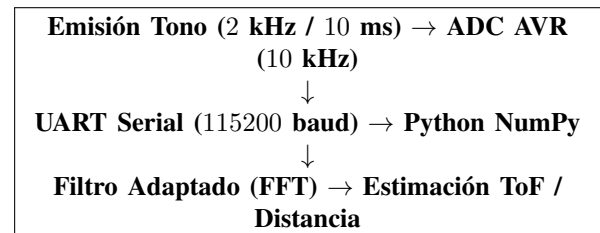


Figura 1. Diagrama de flujo operacional de las herramientas involucradas.

II-A. Adquisición Sincronizada en C++

En la plataforma empotrada, la función `tone(9, 2000)` genera el pulso de sondeo. Simultáneamente, mediante un bucle determinista de microsegundos (`micros()`), se toman $N = 600$ muestras (60 ms totales) a un intervalo $T_s = 100 \mu s$. Las muestras digitalizadas se envían vía UART a 115200 baudios.

II-B. Procesamiento Digital en Python

El script en Python lee el flujo serial y ejecuta el algoritmo de estimación mediante el siguiente modelo matemático:

$$y_{centrada}[n] = y[n] - \frac{1}{N} \sum_{i=0}^{N-1} y[i] \quad (1)$$

Posteriormente, se genera el patrón ideal $x_{patrón}[n]$ modulado por una ventana Hanning y se calcula la correlación cruzada mediante el producto de transformadas:

$$R_{yx}[m] = \mathcal{F}^{-1} \{ \mathcal{F}\{y[n]\} \cdot \mathcal{F}^* \{x_{patrón}[n]\} \} \quad (2)$$

II-C. Soporte de IA Generativa en el Desarrollo

La herramienta Gemini AI se aplicó como un colaborador de diseño. Específicamente, se utilizó para:

1. Validar la corrección sintáctica de las operaciones matriciales en NumPy.

2. Ajustar la ventana Hanning para mitigar la fuga espectral (*spectral leakage*).
3. Formatear las ecuaciones matemáticas y tablas de resultados bajo el estándar IEEE.

III. ADAPTACIÓN DE TÉCNICAS, RECURSOS O MÉTODOS

Para lograr que las herramientas seleccionadas funcionaran en el contexto del problema, fue necesario realizar tres adaptaciones clave en el software y hardware:

III-A. Aceleración del Convertidor Analógico a Digital (ADC)

Por defecto, la función `analogRead()` de Arduino utiliza un prescaler de ADC de 128, lo que impone un tiempo de conversión de $\approx 112 \mu s$, haciendo imposible sostener una tasa de muestreo fija de 10 kHz (100 μs). Se modificó directamente el registro de control ADCSRA del ATmega328P para cambiar el prescaler a 64 (0b00000110):

```
1 // Ajuste del prescaler a 64 (f_ADC = 16MHz/64 = 250kHz)
2 // Tiempo de conversión reducido a ~52 microsegundos
3 ADCSRA = (ADCSRA & 0b11111000) | 0b00000110;
```

Listing 1. Adaptación a nivel de registros del ADC AVR.

Esta adaptación liberó suficiente tiempo de procesamiento para realizar la temporización de precisión del bucle sin perder muestras.

III-B. Enmascaramiento Dinámico de Zona Ciega (Blanking)

La cercanía física entre el buzzer y el micrófono produce una saturación por transmisión sólida durante la emisión del pulso (10 ms). Si se aplicara la correlación directa, el pico máximo siempre ocurriría en $m = 0$.

Se adaptó el algoritmo de búsqueda recortando los primeros 105 desfases (10,5 ms), ignorando la ráfaga directa y buscando el pico únicamente en la zona de retorno útil:

```
1 # Mascara para omitir el disparo directo (105 muestras)
2 lag_minimo_eco = 105
3 zona_busqueda = correlacion_positiva[lag_minimo_eco:]
4 lag_detectado = lag_minimo_eco + np.argmax(np.abs(
    zona_busqueda))
```

Listing 2. Adaptación del algoritmo de correlación en Python.

III-C. Insonorización e Impedancia del Blanco Acústico

Para adaptar las limitaciones del receptor MAX4466 frente al ruido ambiental, se modificó el objeto de prueba cubriendo una estructura de cartón con múltiples capas de papel aluminio. Esta adaptación del recurso físico aprovechó la gran diferencia de impedancia acústica entre el aire y el metal, elevando la amplitud del eco reflejado por encima del nivel de ruido del preamplificador.

IV. CONCLUSIONES SOBRE EL USO DE HERRAMIENTAS

La combinación de optimización a nivel de registros en el hardware empotrado, el procesamiento espectral mediante Python/FFT y el uso guiado de herramientas de IA permitió resolver un problema complejo de medición espacial dentro de las restricciones impuestas de costo y tiempo de procesamiento.

V. REFERENCIAS

REFERENCIAS

- [1] Microchip Technology, "ATmega328P 8-bit AVR Microcontroller Datasheet," DS40002061B, 2018.
- [2] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.